

Programación de bases de datos.

Caso práctico

Juan recuerda, de cuando estudió el Ciclo de Desarrollo de Aplicaciones Informáticas, que había muchas tareas que se podían automatizar dentro de la base de datos mediante el uso de un lenguaje de programación, e incluso que se podían programar algunas restricciones a la hora de manipular los datos. **Juan** se lo comenta a **María** y ésta se muestra ilusionada con dicha idea ya que muchas veces repiten el trabajo con la base de datos de juegos on-line que tienen entre manos (consultas, inserciones, etc. que son muy parecidas y que se podrían automatizar).



[Ministerio de Educación](#) (Uso educativo no)

Para ello hablan con **Ada** y ésta les comenta que claro que se puede hacer y que precisamente eso es lo que les toca hacer ahora. **Ada** les dice que para ese



[Ministerio de Educación](#) (Uso educativo no)

propósito existe un lenguaje de programación llamado PL/SQL que permite hacer lo que ellos quieren, así que les pasa un manual para que se lo vayan leyendo y se vayan poniendo manos a la obra con la base de datos de juegos on-line.

Ahora que ya dominas el uso de SQL para la manipulación y consulta de datos, es el momento de dar una vuelta de tuerca adicional para mejorar las aplicaciones que utilicen nuestra base de datos. Para ello nos vamos a centrar en la programación de bases de datos, utilizando el lenguaje PL/SQL. En esta unidad conoceremos qué es PL/SQL, cuál es su sintaxis y veremos cómo podemos sacarle el máximo partido a nuestra base de datos mediante su uso.

Debes conocer

La mayor parte de los ejemplos de esta unidad están basados en el modelo de datos extraído del siguiente caso de estudio:

[Caso de estudio.](#)



[Ministerio de Educación y Formación Profesional](#). (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.- Introducción.

Caso práctico

Juan y María se han puesto a repasar el manual de PL/SQL que les ha pasado Ada. Aunque no han avanzado mucho con el mismo, ya saben a qué se van a enfrentar y los beneficios que pueden obtener del uso del mismo para su aplicación de juegos on-line. Cuando hacen la primera parada de la mañana para tomarse un café, ambos se ponen a comentar las primeras conclusiones que han sacado después de su primer acercamiento a este lenguaje. Ambos están deseosos de seguir avanzando en su aprendizaje y saben que para ello cuentan con la inestimable ayuda de **Ada**.



ITE (Miguel Martínez Monasterio) (Uso educativo nc)

Estarás pensando que si no tenemos bastante con aprender SQL, sino que ahora tenemos que aprender otro lenguaje más que lo único que va a hacer es complicarnos la vida. Verás que eso no es cierto ya que lo más importante, que es el conocimiento de SQL, ya lo tienes. PL/SQL tiene una sintaxis muy sencilla y verás como pronto te acostumbras y luego no podrás vivir sin él.

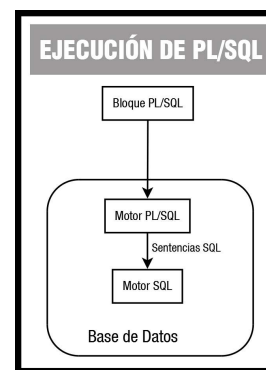
Pero, ¿qué es realmente PL/SQL?

PL/SQL es un lenguaje procedimental estructurado en bloques que amplía la funcionalidad de SQL. Con PL/SQL podemos usar sentencias SQL para manipular datos y sentencias de control de flujo para procesar los datos. Por tanto, PL/SQL combina la potencia de SQL para la manipulación de datos, con la potencia de los lenguajes procedimentales para procesar los datos.

Aunque PL/SQL fue creado por Oracle, hoy día todos los gestores de bases de datos utilizan un lenguaje procedimental muy parecido al ideado por Oracle para poder programar las bases de datos.

Como veremos, en PL/SQL podemos definir variables, constantes, funciones, procedimientos, capturar errores en tiempo de ejecución, anidar cualquier número de bloques, etc. como solemos hacer en cualquier otro lenguaje de programación. Además, por medio de PL/SQL programaremos los disparadores de nuestra base de datos, tarea que no podríamos hacer sólo con SQL.

El motor de PL/SQL acepta como entrada bloques PL/SQL o subprogramas, ejecuta sentencias procedimentales y envía sentencias SQL al servidor de bases de datos. En el esquema adjunto puedes ver su funcionamiento.



Ministerio de Educación (Uso educativo nc)

Una de las grandes ventajas que nos ofrece PL/SQL es un mejor rendimiento en entornos de red cliente-servidor, ya que permite mandar bloques PL/SQL desde el cliente al servidor a través de la red, reduciendo de esta forma el tráfico y así no tener que mandar una a una las sentencias SQL correspondientes.

Para saber más

En el siguiente enlace podrás encontrar una breve historia de PL/SQL.

[Breve historia de PL/SQL.](#)

En estos enlaces podrás comprobar como los gestores de bases de datos incluyen hoy día un lenguaje procedimental para programar la base de datos muy parecido a PL/SQL.

[Procedimientos almacenados en MySQL.](#)

[Lenguaje procedimental en PostgreSQL.](#)

2.- Conceptos básicos.

Caso práctico

Juan ha avanzado muy rápido en la lectura del manual que le pasó **Ada**. **Juan** ya sabe programar en otros lenguajes de programación y por tanto la lectura de los primeros capítulos que se centran en cómo se estructura el lenguaje, los tipos de datos, las estructuras de control, etc. le han resultado muy fáciles de comprender. Sabe que lo único que tendrá que tener en cuenta son algunos aspectos en cuanto a las reglas de escritura y demás, pero la lógica es la de cualquier otro lenguaje de programación.



[ITE](#) (Uso educativo nc)

Como **María** está un poco más verde en el tema de la programación, **Juan** se ha ofrecido a darle un repaso rápido a todo lo que él ha visto y a explicarle todo aquello en lo que tenga dudas y a ver si pronto se pueden poner manos a la obra con la base de datos de juegos on-line.

En este apartado nos vamos a ir introduciendo poco a poco en los diferentes conceptos que debemos tener claros para programar en PL/SQL. Como para cualquier otro lenguaje de programación, debemos conocer las reglas de sintaxis que podemos utilizar, los diferentes elementos de que consta, los tipos de datos de los que disponemos, las estructuras de control que nos ofrece (tanto iterativas como condicionales) y cómo se realiza el manejo de los errores.

Como podrás comprobar, es todo muy sencillo y pronto estaremos escribiendo fragmentos de código que realizan alguna tarea particular. ¡Vamos a ello!

Autoevaluación

Indica cuáles de las siguientes características que nos proporciona PL/SQL son ciertas.

- ☐ Permite reducir el tráfico en la red en entornos cliente-servidor.

- ☐ No podemos utilizar sentencias SQL dentro de un bloque PL/SQL.

- ☐ Nos ofrece las ventajas de SQL y la potencia de un lenguaje procedimental.

- ☐ Para utilizar PL/SQL debemos instalar diferentes drivers en nuestra base de datos Oracle.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Incorrecto

2.1.- Unidades léxicas (I).

En este apartado nos vamos a centrar en conocer cuáles son las unidades léxicas que podemos utilizar para escribir código en PL/SQL. Al igual que en nuestra lengua podemos distinguir diferentes unidades léxicas como palabras, signos de puntuación, etc. En los lenguajes de programación también existen diferentes unidades léxicas que definen los elementos más pequeños que tienen sentido propio y que al combinarlos de manera adecuada, siguiendo las reglas de sintaxis, dan lugar a sentencias válidas sintácticamente.

PL/SQL es un lenguaje no sensible a las mayúsculas, por lo que será equivalente escribir en mayúsculas o minúsculas, excepto cuando hablemos de literales de tipo cadena o de tipo carácter.

Cada unidad léxica puede estar separada por espacios (debe estar separada por espacios si se trata de 2 identificadores), por saltos de línea o por tabuladores para aumentar la legibilidad del código escrito.

```
IF A=CLAVE THEN ENCONTRADO:=TRUE;ELSE ENCONTRADO:=FALSE;END IF;
```

Sería equivalente a escribir la siguiente línea:

```
if a=clave then encontrado:=true;else encontrado:=false;end if;
```

Y también sería equivalente a este otro fragmento que es más legible y facilita su comprensión.

```
IF a = clave THEN
    encontrado := TRUE;
ELSE
    encontrado := FALSE;
END IF;
```

Las unidades léxicas se pueden clasificar en:

- ✔ Delimitadores.
- ✔ Identificadores.
- ✔ Literales.
- ✔ Comentarios.

Vamos a verlas más detenidamente.

Delimitadores.

PL/SQL tiene un conjunto de símbolos denominados delimitadores utilizados para representar operaciones entre tipos de datos, delimitar comentarios, etc. En la siguiente tabla puedes ver un resumen de los mismos.

Delimitadores en PL/SQL.

Delimitadores Simples.		Delimitadores Compuestos.	
Símbolo.	Significado.	Símbolo.	Significado.
+	Suma.	**	Exponenciación.
%	Indicador de atributo.	<>	Distinto.
.	Selector.	!=	Distinto.
/	División.	<=	Menor o igual.
(	Delimitador de lista.	>=	Mayor o igual.
)	Delimitador de lista.	..	Rango.
:	Variable host.		Concatenación.
,	Separador de elementos.	<<	Delimitador de etiquetas.
*	Producto.	>>	Delimitador de etiquetas.
"	Delimitador de identificador acotado.	--	Comentario de una línea.
=	Igual relacional.	/*	Comentario de varias líneas.
<	Menor.	*/	Comentario de varias líneas.
>	Mayor.	:=	Asignación.
@	Indicador de acceso remoto.	=>	Selector de nombre de parámetro.
;	Terminador de sentencias.		
-	Resta/negación.		

2.1.1.- Unidades léxicas (II).

Ya hemos visto qué son los delimitadores. Ahora vamos a continuar viendo el resto de unidades léxicas que nos podemos encontrar en PL/SQL.

Identificadores.

Los identificadores en PL/SQL, como en cualquier otro lenguaje de programación, son utilizados para nombrar elementos de nuestros programas. A la hora de utilizar los identificadores debemos tener en cuenta los siguientes aspectos:

- ✓ Un identificador es una letra seguida opcionalmente de letras, números, \$, _, #.
- ✓ No podemos utilizar como identificador una palabra reservada.
 - Ejemplos válidos: `X`, `A1`, `codigo_postal`.
 - Ejemplos no válidos: `rock&roll`, `on/off`.
- ✓ PL/SQL nos permite además definir los identificadores acotados, en los que podemos usar cualquier carácter con una longitud máxima de 30 y deben estar delimitados por ". Ejemplo: `"X*Y"`.
- ✓ En PL/SQL existen algunos identificadores predefinidos y que tienen un significado especial ya que nos permitirán darle sentido sintáctico a nuestros programas. Estos identificadores son las palabras reservadas y no las podemos utilizar como identificadores en nuestros programas. Ejemplo: `IF`, `THEN`, `ELSE` ...
- ✓ Algunas palabras reservadas para PL/SQL no lo son para SQL, por lo que podríamos tener una tabla con una columna llamada `'type'` por ejemplo, que nos daría un error de compilación al referirnos a ella en PL/SQL. La solución sería acotarlos. `SELECT "TYPE" ...`

Literales.

Los literales se utilizan en las comparaciones de valores o para asignar valores concretos a los identificadores que actúan como variables o constantes. Para expresar estos literales tendremos en cuenta que:

- ✓ Los literales numéricos se expresarán por medio de notación decimal o de notación exponencial. Ejemplos: `234`, `+341`, `2e3`, `-2E-3`, `7.45`, `8.1e3`.
- ✓ Los literales tipo carácter y tipo cadena se deben delimitar con unas comillas simples.
- ✓ Los literales lógicos son `TRUE` y `FALSE`.
- ✓ El literal `NULL` expresa que una variable no tiene ningún valor asignado.

Comentarios.

En los lenguajes de programación es muy conveniente utilizar comentarios en el código. Los comentarios no tienen ningún efecto sobre el código pero sí ayudan mucho al programador o la programadora a recordar qué se está intentando hacer en cada caso (más aún cuando el código es compartido entre varias personas que se dedican a mejorarlo o corregirlo o cuando el mismo autor ha de retomararlo tras mucho tiempo).

En PL/SQL podemos utilizar dos tipos de comentarios:

- ✓ Los comentarios de una línea se expresan por medio del delimitador `--`. Ejemplo:

```
a:=b;  --asignación
```

- ✓ Los comentarios de varias líneas se acotarán por medio de los delimitadores `/*` y `*/`. Ejemplo:

```
/* Primera línea de comentarios.  
Segunda línea de comentarios. */
```

Ejercicio resuelto

Dada la siguiente línea de código, haz su descomposición en las diferentes unidades léxicas que contenga.

```
IF A <> B  
THEN iguales := FALSE; --No son iguales
```

Mostrar retroalimentación

La descomposición en unidades léxicas sería la siguiente:

- ✔ Identificadores: A, B, **iguales**.
- ✔ Identificadores (palabras reservadas): IF, THEN.
- ✔ Delimitadores: <>, :=, ;.
- ✔ Comentarios: --No son **iguales**.

2.2.- Tipos de datos simples, variables y constantes.

En cualquier lenguaje de programación, las variables y las constantes tienen un tipo de dato asignado (bien sea explícitamente o implícitamente). Dependiendo del tipo de dato el lenguaje de programación sabrá la estructura que utilizará para su almacenamiento, las restricciones en los valores que puede aceptar, la precisión del mismo, etc.



[ITE](#) (Uso educativo no)

En PL/SQL contamos con todos los **tipos de datos simples** utilizados en SQL y algunos más. En este apartado vamos a enumerar los más utilizados.

Numéricos.

- ✓ **BINARY_INTEGER**: Tipo de dato numérico cuyo rango es de -2147483647 .. 2147483647. PL/SQL además define algunos subtipos de éste: **NATURAL**, **NATURALN**, **POSITIVE**, **POSITIVEN**, **SIGNTYPE**.
- ✓ **NUMBER**: Tipo de dato numérico para almacenar números racionales. Podemos especificar su escala (-84 .. 127) y su precisión (1 .. 38). La escala indica cuándo se redondea y hacia dónde. Ejemplos. escala=2: 8.234 -> 8.23, escala=-3: 7689 -> 8000. Si se define un **NUMBER(6,2)** estamos indicando que son 6 dígitos numéricos de los cuales 2 son decimales. PL/SQL también define algunos subtipos como: **DEC**, **DECIMAL**, **DOUBLE PRECISION**, **FLOAT**, **INTEGER**, **INT**, **NUMERIC**, **REAL**, **SMALLINT**.
- ✓ **PLS_INTEGER**: Tipo de datos numérico cuyo rango es el mismo que el del tipo de dato **BINARY_INTEGER**, pero que su representación es distinta por lo que las operaciones aritméticas llevadas a cabo con los mismos serán más eficientes que en los dos casos anteriores.

Alfanuméricos.

- ✓ **CHAR(n)**: Array de n caracteres, máximo 2000 bytes. Si no especificamos longitud sería 1. Ocupa en memoria n caracteres.
- ✓ **LONG**: Array de caracteres con un máximo de 32760 bytes.
- ✓ **RAW**: Array de bytes con un número máximo de 2000.
- ✓ **LONG RAW**: Array de bytes con un máximo de 32760.
- ✓ **VARCHAR2(n)**: Tipo de dato para almacenar cadenas de longitud variable con un máximo de 32760. Ocupa en memoria hasta n caracteres.

Grandes objetos.

- ✓ **BFILE**: Puntero a un fichero del Sistema Operativo.
- ✓ **BLOB**: Objeto binario con una capacidad de 4 GB.
- ✓ **CLOB**: Objeto carácter con una capacidad de 2 GB.

Otros.

- ✓ **BOOLEAN**: TRUE/FALSE.
- ✓ **DATE**: Tipo de dato para almacenar valores día/hora desde el 1 enero de 4712 a.c. hasta el 31 diciembre de 4712 d.c.

Hemos visto los tipos de datos simples más usuales. Los tipos de datos compuestos los dejaremos para posteriores apartados.

Atributo %TYPE.

Las variables PL/SQL se suelen declarar para sostener y manipular los datos almacenados en una Base de Datos. Cuando declares variables PL/SQL para guardar los valores de columnas, debes asegurarte de que la variable tenga la precisión (tamaño) y el tipo de dato correcto. Si no es así se puede producir un error durante la ejecución.

Cuando se utiliza el atributo %TYPE para declarar una variable estamos indicando que tiene el mismo tipo de datos que la variable especificada. Normalmente se relaciona con una columna de la BD indicando el nombre de la tabla de base de datos y el nombre de la columna. Si se hace referencia a una variable declarada anteriormente, se indicará el nombre de la variable previamente declarada a la variable por declarar.

Una ventaja del uso de este atributo, es que si cambia la definición de una columna de la BD, no será necesario cambiar la declaración de la variable.

Ejemplos:

```
DECLARE
```

```
nombre_oficina oficinas.nombre%type; -- la variable nombre_oficina tomará el mi  
  
vx number(5,2); -- Variable numérica de 5 dígitos, dos de los cuales son decima  
  
vy vx%type; -- la variable vy tomará el mismo tipo de dato que tenga la variabl
```

Hemos visto los tipos de datos simples más usuales. Los tipos de datos compuestos los dejaremos para posteriores apartados.

Para saber más

En el siguiente enlace podrás ampliar información sobre los tipos de datos de los que disponemos en PL/SQL.

[Tipos de datos en PL/SQL.](#)

Autoevaluación

En PL/SQL cuando vamos a trabajar con enteros es preferible utilizar el tipo de dato `BINARY_INTEGER`, en vez de `PLS_INTEGER`.

- ☐ Verdadero.
- ☐ Falso.

No es correcto, ya que el tipo de dato `PLS_INTEGER` hace que nuestros programas sean más eficientes debido a la representación interna del mismo.

Efectivamente, nuestros programas serán más eficientes al utilizar este tipo de dato debido a su representación interna.

Solución

1. Incorrecto
2. Opción correcta

2.2.1.- Subtipos.

Cuántas veces no has deseado cambiarle el nombre a las cosas por alguno más común para ti. Precisamente, esa es la posibilidad que nos ofrece PL/SQL con la utilización de los subtipos.



[ITE](#) (Uso educativo no)

PL/SQL nos permite definir subtipos de tipos de datos para darles un nombre diferente y así aumentar la legibilidad de nuestros programas. Los tipos de operaciones aplicables a estos subtipos serán las mismas que los tipos de datos de los que proceden. La sintaxis será:

```
SUBTYPE subtipo IS tipo_base;
```

Donde subtipo será el **nombre** que le demos a nuestro subtipo y **tipo_base** será cualquier tipo de dato en PL/SQL.

A la hora de especificar el tipo base, podemos utilizar el modificador **%TYPE** para indicar el tipo de dato de una variable o de una columna de la base de datos y **%ROWTYPE** para especificar el tipo de un cursor o tabla de una base de datos.

```
SUBTYPE id_familia IS familias.identificador%TYPE; -- Permite nombrar a la columna identificador  
SUBTYPE agente IS agentes%ROWTYPE; -- Permite nombrar a las filas de la tabla agentes con el nomb
```

Los subtipos no podemos restringirlos, pero podemos usar un truco para conseguir el mismo efecto y es por medio de una variable auxiliar:

```
SUBTYPE apodo IS varchar2(20);           --ilegal  
aux varchar2(20);  
SUBTYPE apodo IS aux%TYPE;               --legal
```

Los subtipos son intercambiables con su tipo base. También son intercambiables si tienen el mismo tipo base o si su tipo base pertenece a la misma familia:

```
DECLARE  
    SUBTYPE numero IS NUMBER;  
    numero_tres_digitos NUMBER(3);  
    mi_numero_de_la_suerte numero;  
    SUBTYPE encontrado IS BOOLEAN;  
    SUBTYPE resultado IS BOOLEAN;  
    lo_he_encontrado encontrado;  
    resultado_busqueda resultado;  
    SUBTYPE literal IS CHAR;  
    SUBTYPE sentencia IS VARCHAR2;
```

```

literal_nulo literal;
sentencia_vacia sentencia;
BEGIN
...
numero_tres_digitos := mi_numero_de_la_suerte;      --legal
...
lo_he_encontrado := resultado_busqueda;              --legal
...
sentencia_vacia := literal_nulo;                     --legal
...
END;

```

Autoevaluación

Indica la afirmación correcta.

- ☐ Los subtipos lo único que hacen es añadir complejidad a nuestros programas.
- ☐ No hay manera de restringir los subtipos con respecto a su tipo base.
- ☐ Podemos definir un subtipo cuyo tipo base sea una tabla de la base de datos.
- ☐ Podemos definir un subtipo de una variable pero no de una columna de la base de datos.

Incorrecto. Los subtipos añaden legibilidad y no complejidad.

No es cierto. Los subtipos podemos restringirlos con respecto a su tipo base de forma indirecta.

Efectivamente, veo que lo estás entendiendo.

No es correcto. Los subtipos pueden ser de una variable y también de una columna de la base de datos.

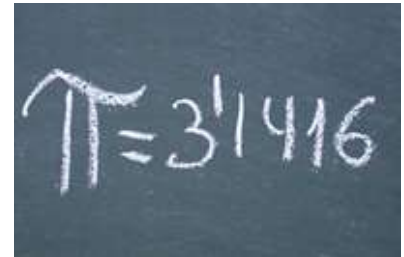
Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto



2.2.2.- Variables y constantes.

Llevamos un buen rato hablando de tipos de datos, variables e incluso de constantes y te estarás preguntando cuál es la forma adecuada de definirlos. En este apartado vamos a ver las diferentes posibilidades a la hora de definirlos y dejaremos para el apartado siguiente ver cuál es el lugar adecuado para hacerlo dentro de un bloque PL/SQL.



[ITE](#) (Uso educativo no)

Para declarar variables o constantes pondremos el nombre de la variable, seguido del tipo de datos y opcionalmente una asignación con el operador `:=`. Si es una constante antepondremos la palabra **CONSTANT** al tipo de dato (lo que querrá decir que no podemos cambiar su valor). Podremos sustituir el operador de asignación en las declaraciones por la palabra reservada **DEFAULT**. También podremos forzar a que no sea nula utilizando la palabra **NOT NULL** después del tipo y antes de la asignación. Si restringimos una variable con **NOT NULL** deberemos asignarle un valor al declararla, de lo contrario PL/SQL lanzará la excepción **VALUE_ERROR** (no te asustes que más adelante veremos lo que son las excepciones, pero como adelanto te diré que es un error en tiempo de ejecución).

```
id SMALLINT;  
hoy DATE := sysdate;  
pi CONSTANT REAL:= 3.1415;  
id SMALLINT NOT NULL; --ilegal, no está inicializada  
id SMALLINT NOT NULL := 9999; --legal  
nombre varchar2(30):= 'Alejandro Magno';  
num INTEGER DEFAULT 4; -- también se puede utilizar DEFAULT para inicializar una variable
```

El alcance y la visibilidad de las variables en PL/SQL será el típico de los lenguajes estructurados basados en bloques, aunque eso lo veremos más adelante.

Conversión de tipos.

Aunque en PL/SQL existe la conversión implícita de tipos para tipos parecidos, siempre es aconsejable utilizar la conversión explícita de tipos por medio de funciones de conversión (**TO_CHAR**, **TO_DATE**, **TO_NUMBER**, ...) y así evitar resultados inesperados.

Precedencia de operadores.

Al igual que en nuestro lenguaje matemático se utiliza una precedencia entre operadores a la hora de realizar las operaciones aritméticas, en PL/SQL también se establece dicha precedencia para evitar confusiones. Si dos operadores tienen la misma precedencia, se evalúa de izquierda a derecha, pero lo aconsejable es utilizar los paréntesis (al igual que hacemos en nuestro lenguaje matemático) para alterar la precedencia de los mismos ya que las operaciones encerradas entre paréntesis tienen mayor precedencia. En la tabla siguiente se muestra la precedencia de los operadores de mayor a menor.

Precedencia de operadores.

Operador.	Operación.
** , NOT	Exponenciación, negación lógica.

Operador.	Operación.
+, -	Identidad, negación.
*, /	Multiplicación, división.
+, -,	Suma, resta y concatenación.
=, !=, <, >, <=, >=, IS NULL, LIKE, BETWEEN, IN	Comparaciones.
AND	Conjunción lógica
OR	Disyunción lógica.

Autoevaluación

Rellena el hueco con el resultado de las siguientes operaciones.

✓ $5+3*2**2$ es igual a: .

✓ $2**3+6/3$ es igual a: .

✓ $2**(3+6/3)$ es igual a: .

Enviar

2.3.- El bloque PL/SQL.

Ya hemos visto las unidades léxicas que componen PL/SQL, los tipos de datos que podemos utilizar y cómo se definen las variables y las constantes. Ahora vamos a ver la unidad básica en PL/SQL que es el bloque.

Un bloque PL/SQL consta de tres zonas:

- ✓ **Declaraciones:** definiciones de variables, constantes, cursores y excepciones.
- ✓ **Proceso:** zona donde se realizará el proceso en sí, conteniendo las sentencias ejecutables.
- ✓ **Excepciones:** zona de manejo de errores en tiempo de ejecución.

La sintaxis es la siguiente:

```
[DECLARE
    [Declaración de variables, constantes, cursores y excepciones]]
BEGIN
    [Sentencias ejecutables]
[EXCEPTION
    Manejadores de excepciones]
END;
```

Los bloques PL/SQL pueden anidarse a cualquier nivel. Como hemos comentado anteriormente el ámbito y la visibilidad de las variables es la normal en un lenguaje procedimental. Por ejemplo, en el siguiente fragmento de código se declara la variable `aux` en ambos bloques, pero en el bloque anidado `aux` con valor igual a 10 actúa de variable global y `aux` con valor igual a 5 actúa como variable local, por lo que en la comparación evaluaría a `FALSE`, ya que al tener el mismo nombre la visibilidad dominante sería la de la variable local.

```
DECLARE
    aux number := 10; -- Variable global
BEGIN
    DECLARE
        aux number := 5; -- Variable local al bloque donde es definida
    BEGIN
        ...
        IF aux = 10 THEN --evalúa a FALSE, no entraría
        ...
    END;
END;
```

Para saber más

En el siguiente enlace podrás ampliar información sobre el ámbito y la visibilidad de las variables en PL/SQL.

[Ámbito y visibilidad en PL/SQL.](#)

Autoevaluación

En PL/SQL el bloque es la unidad básica, por lo que éstos no pueden anidarse.

- ☐ Verdadero.
- ☐ Falso.

No, los bloques se pueden anidar según nos convenga.

Efectivamente, veo que lo vas entendiendo.

Solución

1. Incorrecto
2. Opción correcta

2.4.- Estructuras de control (I).

En la vida constantemente tenemos que tomar decisiones que hacen que llevemos a cabo unas acciones u otras dependiendo de unas circunstancias o repetir una serie de acciones un número dado de veces o hasta que se cumpla una condición. En PL/SQL también podemos imitar estas situaciones por medio de las estructuras de control que son sentencias que nos permiten manejar el flujo de control de nuestro programa, y éstas son dos: condicionales e iterativas.

Control condicional.

Las estructuras de control condicional nos permiten llevar a cabo una acción u otra dependiendo de una condición. Prueba los ejemplos para entender bien el funcionamiento. El caracter / que aparece al final se utiliza para ejecutar el bloque desde SQLPlus.

SENTENCIA IF. Sus variantes son:

- ✓ **Sentencia IF-THEN:** Forma más simple de las sentencias de control condicional. Si la evaluación de la condición es **TRUE**, entonces se ejecuta la secuencia de sentencias encerradas entre el **THEN** y el final de la sentencia.

Sintaxis:

```
IF condicion THEN
secuencia_de_sentencias;
END IF;
```

Ejemplo:

```
SET SERVEROUTPUT ON
DECLARE a integer:=10;
B integer:=7;
BEGIN
IF a>b
  THEN dbms_output.put_line(a || ' es mayor'); -- Como la función put_line solo imprime un valor u
END IF;
END;
/
```

- ✓ **Sentencia IF-THEN-ELSE:** Con esta forma de la sentencia ejecutaremos la primera secuencia de sentencias si la condición se evalúa a **TRUE** y en caso contrario ejecutaremos la segunda secuencia de sentencias.

Sintaxis:

```
IF condicion
THEN Secuencia_de_sentencias1;
```

```
ELSE Secuencia_de_sentencias2;  
END IF;
```

Ejemplo:

```
DECLARE  
a integer:=10;  
b integer:=17;  
BEGIN  
IF a>b THEN  
    dbms_output.put_line(a || ' es mayor');  
ELSE  
    dbms_output.put_line(b || ' es mayor o iguales');  
END IF;  
END;  
/
```

- ✔ **Sentencia IF-THEN-ELSIF:** Con esta última forma de la sentencia condicional podemos hacer una selección múltiple. Si la evaluación de la condición 1 da **TRUE**, ejecutamos la secuencia de sentencias 1, sino evaluamos la condición 2. Si esta evalúa a **TRUE** ejecutamos la secuencia de sentencias 2 y así para todos los **ELSIF** que haya. El último **ELSE** es opcional y es por si no se cumple ninguna de las condiciones anteriores.

Sintaxis:

```
IF condicion1 THEN  
    Secuencia_de_sentencias1;  
ELSIF condicion2 THEN  
    Secuencia_de_sentencias2;  
    ...  
[ELSE  
    Secuencia_de_sentencias;]  
END IF;
```

Ejemplo:

```
IF (operacion = 'SUMA') THEN  
    resultado := arg1 + arg2;  
ELSIF (operacion = 'RESTA') THEN  
    resultado := arg1 - arg2;  
ELSIF (operacion = 'PRODUCTO') THEN  
    resultado := arg1 * arg2;  
ELSIF (arg2 <> 0) AND (operacion = 'DIVISION') THEN  
    resultado := arg1 / arg2;  
ELSE  
    RAISE operacion_no_permitida; -- Lanza un error de ejecución  
END IF;  
/
```

SENTENCIA CASE

Permite representar n sentencias **IF** anidadas, y es más fácil de interpretar cuando se compara con varios valores permitiendo sustituir a las sentencias **IF** encadenadas.

Se puede utilizar de dos formas:

- ✔ Utilizando un selector y un manejador **WHEN** para cada posible valor de ese selector

```
CASE selector
    WHEN expression1 THEN
        ordenes;
    [ WHEN expression2 THEN
        ordenes;
    ...
    [WHEN expression THEN
        ordenes;]
    [ ELSE
        ordenes;]
END CASE ;
```

- ✔ Utilizando condiciones de búsqueda:

```
CASE
    WHEN condición1 THEN
        ordenes;
    [ WHEN condición2 THEN
        ordenes;
    ...
    WHEN condiciónN THEN
        ordenes;]
    [ ELSE
        ordenes;]
END CASE ;
```

En cualquiera de las dos formas, antes de terminar la sentencia **CASE** se puede especificar **ELSE** por si no se ha cumplido ninguna de las condiciones o el valor del selector no ha coincidido con ninguno de los evaluados. Las condiciones, condicion1 a condicionN, son analizadas en el mismo orden en que aparecen listadas y, en el momento en que una de estas condiciones se cumple como verdadera, la sentencia **CASE** devuelve el resultado correspondiente y deja de analizar el resto de condiciones.

Ejemplo:

```
DECLARE
    nota INTEGER:=8; -- Se podría especificar nota INTEGER:=&nota
BEGIN
    CASE
        WHEN nota in(1,2) THEN
            DBMS_OUTPUT.PUT_LINE('Muy deficiente');
        WHEN nota in (3,4) THEN
            DBMS_OUTPUT.PUT_LINE('Insuficiente');
        WHEN nota = 5 THEN
```

```

        DBMS_OUTPUT.PUT_LINE('Suficiente');
    WHEN nota=6 THEN
        DBMS_OUTPUT.PUT_LINE('Bien');
    WHEN nota in(7,8) THEN
        DBMS_OUTPUT.PUT_LINE('Notable');
    WHEN nota in (9,10) THEN
        DBMS_OUTPUT.PUT_LINE('Sobresaliente');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Error, no es una nota');
    END CASE;
END;
/

```

Para pedir datos por teclado en PL/SQL se utilizarán variables de sustitución escribiendo el caracter especial '&' y a continuación un identificador. Si el dato que se va a recibir es una cadena formada por caracteres alfanuméricos, se deben incluir las comillas al definir la expresión.

```

DECLARE
cadena varchar2(25) :='&cad';
BEGIN
    DBMS_OUTPUT.PUT_LINE(cadena);
END;/

```

Autoevaluación

En PL/SQL no existen sentencias que nos permitan tomar una acción u otra dependiendo de una condición.

- ☐ Verdadero.
- ☐ Falso.

Incorrecto, deberías volver a leer este apartado ya que es de lo que trata.

Efectivamente, para eso existen las sentencias de control condicional.

Solución

1. Incorrecto
2. Opción correcta



2.4.1.- Estructuras de control (II). Bucles

Ya que hemos visto las estructuras de control condicional, veamos ahora las estructuras de **control iterativo**.

Control iterativo.

Estas estructuras nos permiten ejecutar una secuencia de sentencias un determinado número de veces.

- ✓ **LOOP:** La forma más simple es el _____ bucle infinito, cuya sintaxis es:

```
LOOP
    secuencia_de_sentencias;
END LOOP;
```

- ✓ **EXIT:** Con esta sentencia forzamos a un bucle a terminar y pasa el control a la siguiente sentencia después del bucle. Un **EXIT** no fuerza la salida de un bloque PL/SQL, sólo la salida del bucle.

```
LOOP
    ...
    IF encontrado = TRUE THEN
        EXIT;
    END IF;
END LOOP;
```

```
DECLARE
    a integer :=1;
BEGIN
    LOOP
        dbms_output.put_line(a);
        IF a>9 THEN
            EXIT;
        END IF;
        a:=a+1;
    END LOOP;
END;
/
```

- ✓ **EXIT WHEN condicion:** Fuerza a salir del bucle cuando se cumple una determinada condición.

```

LOOP
    ...
    EXIT WHEN encontrado;
END LOOP;

```

```

DECLARE
    a integer :=1;
BEGIN
    LOOP
        dbms_output.put_line(a);
        EXIT WHEN a>9;
        a:=a+1;
    END LOOP;
END;
/

```

- ✔ **WHILE LOOP:** Este tipo de bucle ejecuta la secuencia de sentencias mientras la condición sea cierta.

```

WHILE condicion LOOP
    Secuencia_de_sentencias;
END LOOP;

```

```

DECLARE
    a integer :=1;
BEGIN
    WHILE a<10 LOOP
        dbms_output.put_line(a);
        a:=a+1;
    END LOOP;
END;/

```

- ✔ **FOR LOOP:** Este bucle itera mientras el contador se encuentre en el rango definido.

```

FOR contador IN [REVERSE] limite_inferior..limite_superior LOOP
    Secuencia_de_sentencias;
END LOOP;

```

```

DECLARE
    a NUMBER;
BEGIN
    FOR a IN 1..10 LOOP -- ascendente de uno en uno
        dbms_output.put_line(a);
    END LOOP;
    FOR a IN REVERSE 1..10 LOOP -- descendente de uno en uno
        dbms_output.put_line(a);
    END LOOP;

```

Autoevaluación

Al utilizar REVERSE en un bucle FOR, en el rango debemos poner el número mayor el primero y el menor el último.

- ☐ Verdadero.
- ☐ Falso.

No, el rango lo especificamos de misma forma, el menor el primero y el mayor el último, aunque luego se itere del mayor al menor.

Efectivamente, vas por buen camino.

Solución

1. Incorrecto
2. Opción correcta

2.5.- Manejo de errores (I).

Muchas veces te habrá pasado que surgen situaciones inesperadas con las que no contabas y a las que tienes que hacer frente. Pues cuando programamos con PL/SQL pasa lo mismo, que a veces tenemos que manejar errores debidos a situaciones diversas. Vamos a ver cómo tratarlos.

Cualquier situación de error es llamada **excepción** en PL/SQL. Cuando se detecta un error, una excepción es lanzada, es decir, la ejecución normal se para y el control se transfiere a la parte de manejo de excepciones. La parte de manejo de excepciones es la parte etiquetada como **EXCEPTION** y constará de sentencias para el manejo de dichas excepciones, llamadas **manejadores de excepciones**.



[ITE \(Félix Vallés Calvo\)](#) (Uso educativo
nc)

Manejadores de excepciones

Sintaxis:

```
WHEN nombre_excepcion THEN
    <sentencias para su manejo>
    ....
WHEN OTHERS THEN
    <sentencias para su manejo>
```

Ejemplo:

```
DECLARE
    supervisor agentes%ROWTYPE;
BEGIN
    SELECT * INTO supervisor FROM agentes
    WHERE categoria = 2 AND oficina = 3;
    ...
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        --Manejamos el no haber encontrado datos
    WHEN OTHERS THEN
        --Manejamos cualquier error inesperado
END;
/
```

La parte **OTHERS** captura cualquier excepción no considerada.

Las excepciones pueden estar definidas por el usuario o definidas internamente. Las excepciones predefinidas se lanzarán automáticamente asociadas a un error de Oracle. Las excepciones definidas por el usuario deberán definirse y lanzarse explícitamente.

En PL/SQL nosotros podemos definir nuestras propias excepciones en la parte **DECLARE** de cualquier bloque. Estas excepciones podemos lanzarlas explícitamente por medio de la sentencia **RAISE nombre_excepción**.

Excepciones definidas por el usuario

Sintaxis:

```
DECLARE
    nombre_excepcion EXCEPTION;
BEGIN
    ...
    RAISE nombre_excepcion;
    ...
END;
```

Ejemplo:

```
DECLARE
    categoria_erronea EXCEPTION;
BEGIN
    ...
    IF categoria<0 OR categoria>3 THEN
        RAISE categoria_erronea;
    END IF;
    ...
EXCEPTION
    WHEN categoria_erronea THEN
        --manejamos la categoria errónea
END;
```

Debes conocer

En el siguiente enlace podrás ver las diferentes excepciones predefinidas en Oracle, junto a su código de error asociado (que luego veremos lo que es) y una explicación de cuándo son lanzadas.

[Excepciones predefinidas en Oracle.](#)

2.5.1.- Manejo de errores (II).

Ahora que ya sabemos lo que son las excepciones, cómo capturarlas y manejarlas y cómo definir y lanzar las nuestras propias. Es la hora de comentar algunos detalles sobre el uso de las mismas.

- ✓ El alcance de una excepción sigue las mismas reglas que el de una variable, por lo que si nosotros redefinimos una excepción que ya es global para el bloque, la definición local prevalecerá y no podremos capturar esa excepción a menos que el bloque en la que estaba definida esa excepción fuese un bloque nombrado, y podremos capturarla usando la sintaxis: **nombre_bloque.nombre_excepcion**.
- ✓ Las excepciones predefinidas están definidas globalmente. No necesitamos (ni debemos) redefinir las excepciones predefinidas.

```
DECLARE
    no_data_found EXCEPTION;
BEGIN
    SELECT * INTO ...
EXCEPTION
    WHEN no_data_found THEN      --captura la excepción local, no
                                --la global
END;
```

- ✓ Cuando manejamos una excepción no podemos continuar por la siguiente sentencia a la que la lanzó.

```
DECLARE
    ...
BEGIN
    ...
    INSERT INTO familias VALUES
(id_fam, nom_fam, NULL, oficina);
    INSERT INTO agentes VALUES
(id_ag, nom_ag, login, password, 0, 0, id_fam, NULL);
    ...
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        --manejamos la excepción debida a que el nombre de
        --la familia ya existe, pero no podemos continuar por
        --el INSERT INTO agentes, a no ser que lo pongamos
        --explícitamente en el manejador
END;
```

- ✓ Pero sí podemos encerrar la sentencia dentro de un bloque, y ahí capturar las posibles excepciones, para continuar con las siguientes sentencias.

```

DECLARE
    id_fam NUMBER;
    nom_fam VARCHAR2(40);
    oficina NUMBER;
    id_ag NUMBER;
    nom_ag VARCHAR2(60);
    usuario VARCHAR2(20);
    clave VARCHAR2(20);
BEGIN
    ...
    BEGIN
        INSERT INTO familias VALUES (id_fam, nom_fam, NULL, oficina);
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
SELECT identificador INTO id_fam FROM familias WHERE nombre = nom_fam;
    END;
    INSERT INTO agentes VALUES (id_ag, nom_ag, login, password, 1, 1, id_fam, null);
    ...
END;

```

Ejercicio resuelto

Supongamos que queremos reintentar una transacción hasta que no nos dé ningún error. Para ello deberemos encapsular la transacción en un bloque y capturar en éste las posibles excepciones. El bloque lo metemos en un bucle y así se reintentará la transacción hasta que sea posible llevarla a cabo.

Mostrar retroalimentación

```

DECLARE
    id_fam NUMBER;
    nombre VARCHAR2(40);
    oficina NUMBER;
BEGIN
    ...
    LOOP
        BEGIN
            SAVEPOINT inicio;
            INSERT INTO familias VALUES
(id_fam, nombre, NULL, oficina);
            ...
            COMMIT;
        EXIT;
        EXCEPTION
            WHEN DUP_VAL_ON_INDEX THEN
                ROLLBACK TO inicio;

```

```
                                id_fam := id_fam + 1;  
                                END;  
                                END LOOP;  
                                ...  
END;
```

2.5.2.- Manejo de errores (III).

Continuemos viendo algunos detalles a tener en cuenta, relativos al uso de las excepciones.

- ✓ Cuando ejecutamos varias sentencias seguidas del mismo tipo y queremos capturar alguna posible excepción debida al tipo de sentencia, podemos encapsular cada sentencia en un bloque y manejar en cada bloque la excepción, o podemos utilizar una variable localizadora para saber qué sentencia ha sido la que ha lanzado la excepción (aunque de esta manera no podremos continuar por la siguiente sentencia).



[ITE \(Elena Hervás\)](#) (Uso educativo nc)

```
DECLARE
    sentencia NUMBER := 0;
BEGIN
    ...
    SELECT * FROM agentes ...
    sentencia := 1;
    SELECT * FROM familias ...
    sentencia := 2;
    SELECT * FROM oficinas ...
    ...
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        IF sentencia = 0 THEN
            RAISE agente_no_encontrado;
        ELSIF sentencia = 1 THEN
            RAISE familia_no_encontrada;
        ELSIF sentencia = 2 THEN
            RAISE oficina_no_encontrada;
        END IF;
END;/
```

- ✓ Si la excepción es capturada por un manejador de excepción apropiado, ésta es tratada y posteriormente el control es devuelto al bloque superior. Si la excepción no es capturada y no existe bloque superior, el control se devolverá al entorno. También puede darse que la excepción sea manejada en un bloque superior a falta de manejadores para ella en los bloques internos, la excepción se propaga de un bloque al superior y así hasta que sea manejada o no queden bloques superiores con lo que el control se devuelve al entorno. Una excepción también puede ser relanzada en un manejador. En la siguiente presentación puedes ver cómo se propagan diferentes excepciones entre diferentes bloques.

[Propagación de excepciones en PL/SQL.](#) (odp - 17,90 KB)
[Resumen textual alternativo](#)

Autoevaluación

Todas las excepciones están predefinidas y nosotros no podemos definir nuevas excepciones.

- ☐ Verdadero.
- ☐ Falso.

No, también podemos definir nuestras propias excepciones.

Efectivamente, esta afirmación es falsa.

Solución

1. Incorrecto
2. Opción correcta

Las excepciones definidas por el usuario deben ser lanzadas explícitamente.

- ☐ Verdadero.
- ☐ Falso.

Efectivamente, las lanzamos explícitamente usando la sentencia **RAISE**.

No, deberías repasar los contenidos.

Solución

1. Opción correcta
2. Incorrecto

Es obligatorio declarar todas las excepciones predefinidas que vamos a usar en nuestros bloques.

- ☐ Verdadero.
- ☐ Falso.

No y además no es aconsejable hacerlo.

Efectivamente, ni es obligatorio ni debemos hacerlo.

Solución

1. Incorrecto
2. Opción correcta

2.5.3.- Manejo de errores (IV).

Oracle también permite que nosotros lancemos nuestros propios mensajes de error a las aplicaciones y asociarlos a un código de error que Oracle reserva, para no interferir con los demás códigos de error. Lo hacemos por medio del procedimiento:

```
RAISE_APPLICATION_ERROR(error_number, message [, (TRUE|FALSE)]);
```

Donde **error_number** es un entero negativo comprendido entre -20000..-20999 y **message** es una cadena que devolvemos a la aplicación. El tercer parámetro especifica si el error se coloca en la pila de errores (**TRUE**) o se vacía la pila y se coloca únicamente el nuestro (**FALSE**). Sólo podemos llamar a este procedimiento desde un subprograma.

No hay excepciones predefinidas asociadas a todos los posibles errores de Oracle, por lo que nosotros podremos asociar excepciones definidas por nosotros a errores Oracle, por medio de la directiva al compilador (o pseudoinstrucción):

```
PRAGMA_INIT( nombre_excepcion, error_Oracle )
```

Donde **nombre_excepcion** es el nombre de una excepción definida anteriormente, y **error_Oracle** es el número negativo asociado al error.

```
DECLARE
    no_null EXCEPTION;
    PRAGMA EXCEPTION_INIT(no_null, -1400);
    id familias.identificador%TYPE;
    nombre familias.nombre%TYPE;
BEGIN
    ...
    nombre := NULL;
    ...
    INSERT INTO familias VALUES (id, nombre, null, null);
EXCEPTION
    WHEN no_null THEN
        ...
END;
```

Oracle asocia 2 funciones para comprobar la ejecución de cualquier sentencia. **SQLCODE** nos devuelve el código de error y **SQLERRM** devuelve el mensaje de error asociado. Si una sentencia es ejecutada correctamente, **SQLCODE** nos devuelve 0 y en caso contrario devolverá un número negativo asociado al error (excepto **NO_DATA_FOUND** que tiene asociado el +100).

```
DECLARE
    cod number;
    msg varchar2(100);
BEGIN
    ...
EXCEPTION
```

```
WHEN OTHERS THEN
  cod := SQLCODE;
  msg := SUBSTR(SQLERRM, 1, 1000);
  INSERT INTO errores VALUES (cod, msg);
END;
```

Autoevaluación

De las siguientes afirmaciones marca las que creas que son correctas.

- ☐ Podemos lanzar nuestros propios mensajes de error a las aplicaciones.
- ☐ Podemos acceder al código de error generado por la ejecución de una sentencia pero no a su mensaje asociado.
- ☐ Podemos asociar excepciones definidas por nosotros a códigos de error de Oracle.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto

2.6.- Sentencias SQL en programas PL/SQL

En un bloque PL/SQL, se utilizan sentencias SQL para recuperar y modificar los datos de tablas de una BD.

Hasta ahora hemos trabajado con SQL de forma interactiva, escribíamos la sentencia y obteníamos los resultados en pantalla. Para trabajar con SQL dentro de un programa trabajaremos con SQL embebido o incrustado. Los datos devueltos de la consulta los guardaremos en variables y estructuras definidas en el lenguaje anfitrión, en nuestro caso PLSQL, para utilizarlas como convenga. Para ello añadiremos alguna cláusula a la sentencia **SELECT**.

PL/SQL soporta el Lenguaje de Manipulación de Datos (DML) y los comandos de control de transacciones. Se pueden utilizar los comandos DML para modificar los datos de una tabla de Base de Datos.

PL/SQL no soporta directamente el Lenguaje de Definición de Datos (DDL) como **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**, etc... Lo cual no puede suceder si las aplicaciones tienen que crear objetos de Base de Datos en tiempo de ejecución mediante el paso de valores. Las sentencias DDL no pueden ser ejecutadas directamente. Estas instrucciones son sentencias dinámicas de SQL. Las sentencias dinámicas de SQL se construyen como cadenas de caracteres en tiempo de ejecución y pueden contener marcadores de posición para parámetros. Por lo tanto, puede utilizar SQL dinámico para ejecutar sus sentencias DDL en PL/SQL.

PL/SQL no soporta directamente las sentencias de Lenguaje de Control de Datos (DCL), tales como **GRANT** o **REVOKE**. Puede utilizar SQL dinámico para ejecutarlas.

Si queremos recuperar datos de la BD utilizaremos la instrucción **SELECT** utilizaremos este formato.

Sintaxis:

```
SELECT lista_campos INTO {nombre_variable[, nombre_variable]...| nombre_registro}
FROM tabla
[WHERE condición];
```

donde

- la lista_campos, contendrá al menos una columna y puede incluir expresiones SQL, funciones de fila, o funciones de grupo.
- la cláusula **INTO** es obligatoria y se especifica entre **SELECT** y cláusula **FROM**.
- nombre_variable, variable donde se guardará el valor recuperado. Deben especificarse tantas variables como campos se indiquen en la lista de campos, debe haber correspondencia en posición y en tipo de datos.
- nombre_registro, un dato compuesto de PL/SQL donde se guardarán los valores recuperados

Las instrucciones **SELECT** dentro de un programa PLSQL deben devolver una sola fila. Una consulta que devuelve más de una fila o ninguna fila genera un error de tipo, respectivamente **TOO_MANY_ROWS** o **NO_DATA_FOUND**. Si queremos recuperar más de una fila necesitaremos una estructura de datos llamada cursor que se verá más adelante.

Las instrucciones DML, **INSERT**, **UPDATE** y **DELETE**, se ejecutan utilizando la misma sintaxis que en SQL.

3.- Tipos de datos compuestos.

Caso práctico

María, gracias a la ayuda de Juan, ha comprendido muy bien el manejo básico de PL/SQL. **Juan** le comenta que en la mayoría de los lenguajes de programación, además de los tipos de datos simples, existen tipos de datos compuestos que le dan mucha más versatilidad a los mismos y una gran potencia. **Juan** no conoce bien si PL/SQL dispone de estos tipos de datos y si dispone de ellos ¿de cuáles?



[Ministerio de Educación](#) (Uso educativo nc)

María y **Juan** van a hablar con **Ada** para ver si les puede aclarar un poco la situación y dar unas pequeñas pautas para empezar. **Ada** les comenta que claro que PL/SQL cuenta con este tipo de datos, pero que hoy tiene una reunión importantísima y que tiene que terminar de preparársela, por lo que los remite al capítulo sobre tipos de datos compuestos del manual que les pasó. **Juan** y **María** le dicen que no se preocupe y que ya verá como a la vuelta de esa reunión son capaces de saber cuáles son e incluso de dominarlos. **Ada**, para motivarlos, les dice que si es así los invita a un aperitivo a su vuelta. Así que **Juan** y **María** se ponen con el capítulo de tipos de datos compuestos del manual para ver si pueden sorprender a **Ada** a su vuelta.

En el capítulo anterior, entre otras cosas, hemos conocido los tipos de datos simples con los que cuenta PL/SQL. Pero dependiendo de la complejidad de los problemas, necesitamos disponer de otras estructuras en las que apoyarnos para poder modelar nuestro problema. En este capítulo nos vamos a centrar en conocer los tipos de datos complejos que nos ofrece PL/SQL y cómo utilizarlos para así poder sacarle el mayor partido posible.

Citas para pensar

"Todo lo complejo puede dividirse en partes simples".

René Descartes.

3.1.- Registros.

El uso de los registros es algo muy común en los lenguajes de programación. PL/SQL también nos ofrece este tipo de datos. En este apartado veremos qué son y cómo definirlos y utilizarlos.

Un **registro** es un grupo de elementos relacionados, referenciados por un único nombre, almacenados en campos, cada uno de los cuales tiene su propio nombre y tipo de dato.

Por ejemplo, una dirección podría ser un registro con campos como calle, número, piso, puerta, código postal, ciudad, provincia y país. Los registros hacen que la información sea más fácil de organizar y representar. Para declarar un registro seguiremos la siguiente sintaxis:

```
TYPE nombre_tipo IS RECORD (decl_campo[, decl_campo] ...);
```

donde:

```
decl_campo := nombre_tipo [[NOT NULL] {:=|DEFAULT} expresion]
```

El tipo del campo será cualquier tipo de dato válido en PL/SQL excepto **REF CURSOR**. La expresión será cualquier expresión que evalúe al tipo de dato del campo.

```
TYPE direccion IS RECORD
(
  calle          VARCHAR2(50),
  numero         INTEGER(4),
  piso           INTEGER(4),
  puerta         VARCHAR2(2),
  codigo_postal  INTEGER(5),
  ciudad         VARCHAR2(30),
  provincia      VARCHAR2(20),
  pais           VARCHAR2(20) := 'España'
);
mi_direccion direccion;
```

Para acceder a los campos usaremos el operador punto.

```
...
mi_direccion.calle := 'Ramirez Arellano';

mi_direccion.numero := 15;

...
```

Para asignar un registro a otro, éstos deben ser del mismo tipo, no basta que tengan el mismo número de campos y éstos emparejen uno a uno. Tampoco podemos comparar registros aunque sean del mismo tipo, ni tampoco comprobar si éstos son nulos. Podemos hacer **SELECT** en registros, pero no podemos hacer **INSERT** desde registros.

```
DECLARE
TYPE familia IS RECORD
(
    identificador    NUMBER,
    nombre            VARCHAR2(40),
    padre            NUMBER,
    oficina           NUMBER
);
TYPE familia_aux IS RECORD
(
    identificador    NUMBER,
    nombre            VARCHAR2(40),
    padre            NUMBER,
    oficina           NUMBER
);
SUBTYPE familia_fila IS familias%ROWTYPE; -- tendrá los mismos campos que tenga la tabla fam:
mi_fam familia;
mi_fam_aux familia_aux;
mi_fam_fila familia_fila;
BEGIN
...
mi_fam := mi_fam_aux;                --ilegal
mi_fam := mi_fam_fila;               --legal
IF mi_fam IS NULL THEN ...           --ilegal
IF mi_fam = mi_fam_fila THEN ...     --ilegal
SELECT * INTO mi_fam FROM familias ... --legal
INSERT INTO familias VALUES (mi_fam_fila); --ilegal
...
END;/
```

Autoevaluación

Un registro se puede asignar a otro siempre que tenga el mismo número de campos y éstos emparejen uno a uno.

- ☐ Verdadero.
- ☐ Falso.

No, deben ser del mismo tipo para poder asignarlos.

Efectivamente, veo que lo vas entendiendo.

Solución

1. Incorrecto
2. Opción correcta

3.2.- Colecciones. Arrays de longitud variable.

Una colección es un grupo ordenado de elementos, todos del mismo tipo. Cada elemento tiene un subíndice único que determina su posición en la colección.

Una colección es un grupo ordenado de elementos, todos del mismo tipo. Cada elemento tiene un subíndice único que determina su posición en la colección.

En PL/SQL las colecciones sólo pueden tener una dimensión. PL/SQL ofrece 2 clases de colecciones: arrays de longitud variable y tablas anidadas.

Arrays de longitud variable.

Los elementos del tipo **VARRAY** son los llamados arrays de longitud variable. Son como los arrays de cualquier otro lenguaje de programación, pero con la salvedad de que a la hora de declararlos, nosotros indicamos su tamaño máximo y el array podrá ir creciendo dinámicamente hasta alcanzar ese tamaño. Un **VARRAY** siempre tiene un límite inferior igual a 1 y un límite superior igual al tamaño máximo.

Para declarar un **VARRAY** usaremos la sintaxis:

```
TYPE nombre IS {VARRAY | VARYING} (tamaño_máximo) OF tipo_elementos [NOT NULL];
```

Donde **tamaño_máximo** será un entero positivo y **tipo_elementos** será cualquier tipo de dato válido en PL/SQL, excepto **BINARY_INTEGER**, **BOOLEAN**, **LONG**, **LONG RAW**, **NATURAL**, **NATURALN**, **NCHAR**, **NCLOB**, **NVARCHAR2**, objetos que tengan como atributos **TABLE** o **VARRAY**, **PLS_INTEGER**, **POSITIVE**, **POSITIVEN**, **SIGNTYPE**, **STRING**, **TABLE**, **VARRAY**. Si **tipo_elementos** es un registro, todos los campos deberían ser de un tipo escalar.

Cuando definimos un **VARRAY**, éste es automáticamente nulo, por lo que para empezar a utilizarlo deberemos inicializarlo. Para ello podemos usar un constructor:

```
TYPE familias_hijas IS VARRAY(100) OF familia;
familias_hijas1 familias_hijas := familias_hijas( familia(100, 'Fam100', 10, null
```

También podemos usar constructores vacíos.

```
familias_hijas2 familias_hijas := familias_hijas();
```

Para referenciar elementos en un **VARRAY** utilizaremos la sintaxis **nombre_colección(subíndice)**. Si una función devuelve un **VARRAY**, podemos usar la sintaxis: **nombre_funcion(lista_parametros)**

(subíndice).

```
IF familias_hijas1(i).identificador = 100 THEN ...
    IF dame_familias_hijas(10)(i).identificador = 100 THEN ...
```

Un **VARRAY** puede ser asignado a otro si ambos son del mismo tipo.

```
DECLARE

    TYPE tabla1 IS VARRAY(10) OF NUMBER;
    TYPE tabla2 IS VARRAY(10) OF NUMBER;
    mi_tabla1 tabla1 := tabla1();
    mi_tabla2 tabla2 := tabla2();
    mi_tabla tabla1 := tabla1();
BEGIN
    ...
    mi_tabla := mi_tabla1;          --legal
    mi_tabla1 := mi_tabla2;        --ilegal
    ...
END;
```

Para extender un **VARRAY** usaremos el método **EXTEND**. Sin parámetros, extendemos en 1 elemento nulo el **VARRAY**. **EXTEND(n)** añade n elementos nulos al **VARRAY** y **EXTEND(n,i)** añade n copias del i-ésimo elemento.

COUNT nos dirá el número de elementos del **VARRAY**. **LIMIT** nos dice el tamaño máximo del **VARRAY**. **FIRST** siempre será 1. **LAST** siempre será igual a **COUNT**. **PRIOR** y **NEXT** devolverá el antecesor y el sucesor del elemento.

Al trabajar con **VARRAY** podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de los mismos: **COLECTION_IS_NULL**, **SUBSCRIPT_BEYOND_COUNT**, **SUBSCRIPT_OUTSIDE_LIMIT** y **VALUE_ERROR**.

Ejemplos de uso de los **VARRAY**.

✔ Extender un **VARRAY**.

```
DECLARE

    TYPE tab_num IS VARRAY(10) OF NUMBER;
    mi_tab tab_num;

BEGIN

    mi_tab := tab_num();
    FOR i IN 1..10 LOOP
        mi_tab.EXTEND;
        mi_tab(i) := calcular_elemento(i);
    END LOOP;
    ...

END;
```

- Consultar propiedades **VARRAY**.

```

DECLARE
    TYPE numeros IS VARRAY(20) OF NUMBER;
    tabla_numeros numeros := numeros();
    num NUMBER;
BEGIN
    num := tabla_numeros.COUNT; --num := 0
    FOR i IN 1..10 LOOP
        tabla_numeros.EXTEND;
        tabla_numeros(i) := i;
    END LOOP;
    num := tabla_numeros.COUNT; --num := 10
    num := tabla_numeros.LIMIT; --num := 20
    num := tabla_numeros.FIRST; --num := 1;
    num := tabla_numeros.LAST; --num := 10;
    ...
END;

```

- Posibles excepciones.

```

DECLARE
    TYPE numeros IS VARRAY(20) OF INTEGER;
    v_numeros numeros := numeros( 10, 20, 30, 40 );
    v_enteros numeros;
BEGIN
    v_enteros(1) := 15; --lanzaría COLECTION_IS_NULL
    v_numeros(5) := 20; --lanzaría SUBSCRIPT_BEYOND_COUNT
    v_numeros(-1) := 5; --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
    lanzaría VALUE_ERROR
    ....
END;

```

Autoevaluación

Indica, de entre las siguientes, cuál es la afirmación correcta referida a **VARRAY**.

- ☐ Un **VARRAY** no hace falta inicializarlo.
- ☐ **COUNT** y **LIMIT** siempre nos devolverán el mismo valor.
- ☐ **LAST** y **COUNT** siempre nos devolverán el mismo valor.

Incorrecto, sí es necesario inicializar un **VARRAY** para poder empezar a utilizarlo.

No, esta afirmación será cierta en el caso en el que el **VARRAY** esté lleno totalmente.

Efectivamente, estás en lo cierto.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

3.2.1.- Colecciones. Tablas anidadas.

Las tablas anidadas son colecciones de elementos, que no tienen límite superior fijo, y pueden aumentar dinámicamente su tamaño. Además podemos borrar elementos individuales.

Para declararlos utilizaremos la siguiente sintaxis:

```
TYPE nombre IS TABLE OF tipo_elementos [NOT NULL];
```

Donde **tipo_elementos** tendrá las mismas restricciones que para los **VARRAY**.

Al igual que pasaba con los **VARRAY**, al declarar una tabla anidada, ésta es automáticamente nula, por lo que deberemos inicializarla antes de usarla.

```
TYPE hijos IS TABLE OF agente;
hijos_fam hijos := hijos( agente(...) ...);
```

También podemos usar un constructor nulo.

Para referenciar elementos usamos la misma sintaxis que para los **VARRAY**.

Para extender una tabla usamos **EXTEND** exactamente igual que para los **VARRAY**. **COUNT** nos dirá el número de elementos, que no tiene por qué coincidir con **LAST**. **LIMIT** no tiene sentido y devuelve **NULL**. **EXISTS(n)** devuelve **TRUE** si el elemento existe, y **FALSE** en otro caso (el elemento ha sido borrado). **FIRST** devuelve el primer elemento que no siempre será 1, ya que hemos podido borrar elementos del principio. **LAST** devuelve el último elemento. **PRIOR** y **NEXT** nos dicen el antecesor y sucesor del elemento (ignorando elementos borrados). **TRIM** sin argumentos borra un elemento del final de la tabla. **TRIM(n)** borra n elementos del final de la tabla. **TRIM** opera en el tamaño interno, por lo que si encuentra un elemento borrado con **DELETE**, lo incluye para ser eliminado de la colección. **DELETE(n)** borra el n-ésimo elemento. **DELETE(n, m)** borra del elemento n al m. Si después de hacer **DELETE**, consultamos si el elemento existe nos devolverá **FALSE**.

Al trabajar con tablas anidadas podemos hacer que salte alguna de las siguientes excepciones, debidas a un mal uso de las mismas: **COLECTION_IS_NULL**, **NO_DATA_FOUND**, **SUBSCRIPT_BEYOND_COUNT** y **VALUE_ERROR**.

Ejemplos de uso de las tablas anidadas.

Diferentes operaciones sobre tablas anidadas.

```
DECLARE
TYPE numeros IS TABLE OF NUMBER;
tabla_numeros numeros := numeros();
num NUMBER;
BEGIN
num := tabla_numeros.COUNT;      --num := 0
FOR i IN 1..10 LOOP
    tabla_numeros.EXTEND;
    tabla_numeros(i) := i;
END LOOP;
num := tabla_numeros.COUNT;      --num := 10
```

```

tabla_numeros.DELETE(10);
num := tabla_numeros.LAST;      --num := 9
num := tabla_numeros.FIRST;     --num := 1
tabla_numeros.DELETE(1);
num := tabla_numeros.FIRST;     --num := 2
FOR i IN 1..4 LOOP
    tabla_numeros.DELETE(2*i);
END LOOP;
num := tabla_numeros.COUNT;     --num := 4
num := tabla_numeros.LAST;     --num := 9
...
END;

```

Posibles excepciones en su uso.

```

DECLARE
TYPE numeros IS TABLE OF NUMBER;
tabla_num numeros := numeros();
tabla1 numeros;
BEGIN
tabla1(5) := 0;      --lanzaría COLLECTION_IS_NULL
tabla_num.EXTEND(5);
tabla_num.DELETE(4);
tabla_num(4) := 3;   --lanzaría NO_DATA_FOUND
tabla_num(6) := 10;  --lanzaría SUBSCRIPT_BEYOND_COUNT
tabla_num(-1) := 0;  --lanzaría SUBSCRIPT_OUTSIDE_LIMIT
tabla_num('y') := 5;--lanzaría VALUE_ERROR
END;

```

Autoevaluación

Las tablas anidadas podemos hacer que crezcan dinámicamente, pero no podemos borrar elementos.

- ☐ Verdadero.
- ☐ Falso.

No, en las tablas anidadas también podemos borrar elementos.

¡Muy bien!

Solución

1. Incorrecto
2. Opción correcta

3.3.- Cursores.

En los apartados anteriores hemos visto algunos tipos de datos compuestos cuyo uso es común en otros lenguajes de programación. Sin embargo, en este apartado vamos a ver un tipo de dato, que aunque se puede asemejar a otros que ya conozcas, su uso es exclusivo en la programación de las bases de datos y que es el **cursor**.

Un **cursor** no es más que una estructura que almacena el conjunto de filas devuelto por una consulta a la base de datos.

Oracle usa áreas de trabajo para ejecutar sentencias SQL y almacenar la información procesada. Hay 2 clases de cursores: **implícitos** y **explícitos**. PL/SQL declara implícitamente un cursor para todas las sentencias SQL de manipulación de datos, incluyendo consultas que devuelven una sola fila. Para las consultas que devuelven más de una fila, se debe declarar explícitamente un cursor para procesar las filas individualmente.

En este primer apartado vamos a hablar de los cursores implícitos y de los atributos de un cursor (estos atributos tienen sentido con los cursores explícitos, pero los introducimos aquí para ir abriendo boca), para luego pasar a ver los cursores explícitos y terminaremos hablando de los cursores variables.

Cursores implícitos.

Oracle abre implícitamente un cursor para procesar cada sentencia SQL que no esté asociada con un cursor declarado explícitamente.

Con un cursor implícito no podemos usar las sentencias **OPEN**, **FETCH** y **CLOSE** para controlar el cursor. Pero sí podemos usar los atributos del cursor para obtener información sobre las sentencias SQL más recientemente ejecutadas.

Atributos de un cursor.

Cada cursor tiene 4 atributos que podemos usar para obtener información sobre la ejecución del mismo o sobre los datos. Estos atributos pueden ser usados en PL/SQL, pero no en SQL. Aunque estos atributos se refieren en general a cursores explícitos y tienen que ver con las operaciones que hayamos realizado con el cursor, es deseable comentarlas aquí y en el siguiente apartado tomarán pleno sentido.

- ✔ **%FOUND**: Después de que el cursor esté abierto y antes del primer **FETCH**, **%FOUND** devuelve **NULL**. Después del primer **FETCH**, **%FOUND** devolverá **TRUE** si el último **FETCH** ha devuelto una fila y **FALSE** en caso contrario. Para cursores implícitos **%FOUND** devuelve **TRUE** si un **INSERT**, **UPDATE** o **DELETE** afectan a una o más de una fila, o un **SELECT ... INTO ...** devuelve una o más filas. En otro caso **%FOUND** devuelve **FALSE**.
- ✔ **%NOTFOUND**: Es lógicamente lo contrario a **%FOUND**.
- ✔ **%ISOPEN**: Evalúa a **TRUE** si el cursor está abierto y **FALSE** en caso contrario. Para cursores implícitos, como Oracle los cierra automáticamente, **%ISOPEN** evalúa siempre a **FALSE**.
- ✔ **%ROWCOUNT**: Para un cursor abierto y antes del primer **FETCH**, **%ROWCOUNT** evalúa a 0. Después de cada **FETCH**, **%ROWCOUNT** es incrementado y evalúa al número de filas que hemos procesado. Para cursores implícitos **%ROWCOUNT** evalúa al número de filas afectadas por un **INSERT**, **UPDATE** o **DELETE** o el número de filas devueltas por un **SELECT ... INTO ...**.

Debes conocer

Aunque todavía no hemos visto las operaciones que se pueden realizar con un cursor explícito, es conveniente que te vayas familiarizando con la evaluación de sus atributos según las operaciones que hayamos realizado con el cursor y que tomarán pleno sentido cuando veamos el siguiente apartado.

[Evaluación de los atributos de un cursor explícito.](#)